COURSE LAB GUIDE · 32 LABS

Database Lab Guide

Hands-on exercises for every video in the course, from the first SELECT to backups, security, and troubleshooting in production. Every lab runs against one shared environment you build in Lab 02.

How to use this guide

Each lab matches one video. Watch the video, then do the lab the same day while the ideas are fresh. Most labs build on the labs before them, so do them in order. Times are for a first attempt and include reading the output.

Every lab follows the same layout:

SECTION	WHAT IT GIVES YOU
Objective	The single skill the lab trains.
Steps	The exercise, in order. Write down what you expect before you run anything.
Starter code	Enough SQL or shell to get you started. Extend it; don't just paste it.
Checkpoint	How you know the lab worked. Don't move on until this passes.
Break it	A failure on purpose, drawn from the video's failure cases. Cause it, watch what happens, then fix it.
Deliverable	What goes into your lab notebook (labnotes.md).

GROUND RULES

1. **Predict, then run.** Write the row count or result you expect first. If you didn't predict it, you haven't checked it.
2. **Never change shared data without a safety net.** Use a transaction you can roll back or a copy of the table.
3. **Break things freely.** The reset script from Lab 02 puts everything back in under a minute.
4. **Write it down.** A design or a fix that isn't recorded is lost. Every lab ends with a notebook entry.

The lab environment

The course teaches on PostgreSQL, a full relational database used widely in production. Nearly all of the SQL carries over to MySQL, SQL Server, Oracle, and SQLite. Where a lab uses something specific to PostgreSQL, it says so.

COMPONENT	CHOICE	USED IN
Database server	PostgreSQL 16 in Docker	All labs
Query clients	psql, plus DBeaver or pgAdmin	All labs
Sample database	shopfloor (course files)	Labs 02–11, 16, 18–23, 26, 29–31
Design scenarios	toollib, vetclinic (you build these)	Labs 12–17
Python 3.11+	psycopg 3, SQLAlchemy 2	Labs 21, 28, 30
Docker Compose	Multi-container setups	Labs 23, 24, 31
Cloud account	Any provider's managed PostgreSQL, free tier	Lab 27

The shopfloor dataset

shopfloor is a mid-sized online retailer: customers, a product catalog organized as a category tree, orders, and the staff who handle them. Some of the mess is there on purpose (missing discounts, duplicate emails, orders that never shipped), because later labs need it.

TABLE	ROWS	NOTES
`customers`	5,000	Includes a referred_by self-reference and some duplicate emails.
`categories`	42	A tree: parent_id points to another category.
`products`	400	unit_price is numeric(10,2); some products are discontinued.
`employees`	60	manager_id points to another employee.
`orders`	60,000	ordered_at is a timestamptz; shipped_at is null if the order never shipped.
`order_items`	180,000	discount is nullable, on purpose.
`payments`	62,500	Some orders were paid in several parts.

Contents

LAB	TITLE	TIME
01	Find the databases you already use	30 min
02	Build your database lab	60 min
03	First queries with SELECT	60 min
04	Data types, expressions, and NULL	60 min
05	Think in sets	45 min
06	Join tables together	75 min
07	Aggregate and group	60 min
08	Subqueries and nested logic	60 min
09	Window functions and CTEs	75 min
10	Modify data safely	60 min
11	Views, patterns, and query craft	60 min
12	Design before you build	60 min
13	Model relationships	75 min
14	Keys and constraints	60 min
15	Normalize a redundant table	75 min
16	Denormalize on purpose	75 min
17	Design for a messy scenario	120 min
18	Look under the hood	60 min
19	Make a slow query fast with an index	75 min
20	Read execution plans	75 min
21	Transactions and integrity	60 min
22	Concurrency, locking, and isolation	90 min

LAB	TITLE	TIME
23	Replication and read scaling	90 min
24	Relational versus document	75 min
25	Build a database selection framework	45 min
26	Build a small data warehouse	90 min
27	Run a managed database and ship a schema change	120 min
28	Look behind the ORM and split data across services	75 min
29	Back up, restore, and monitor	90 min
30	Least privilege and SQL injection	90 min
31	Troubleshoot staged incidents	120 min
32	Assess your skills and plan the next 90 days	60 min

PART I

Foundations

Vocabulary, a working lab, and the first habits of correct querying.

01 Find the databases you already use

VIDEO 1 · 30 MIN

OBJECTIVE Work out what data a real application has to store, how it gets asked for, and why plain files would fail it.

STEPS

1. Pick five apps you use every day, for example a bank, a messenger, a music service, a map, and a shop.
2. For each one, list the three to five kinds of things it must store (accounts, messages, tracks, places, orders).
3. For each kind of thing, note how the app asks for it: lookup by ID, search, a feed, or a summary such as a monthly total.
4. Mark the one fact per app that must never be wrong (a balance, a delivery address) and the moment two users could change it at once.
5. For each app, write one sentence on what would break if the data lived in a folder of files.

```
App:                 _____________________
Must store:          _____________________
Asked for by:        lookup | search | feed | summary
Must never be wrong: _____________________
Concurrent change:   _____________________
If it were files:    _____________________
```

Worksheet. Copy it once per app.

CHECKPOINT

Every app has at least one "must never be wrong" fact and one concurrency risk, each tied to a real feature you have used.

BREAK IT

Two phones edit the same contacts.csv at the same moment, and both save. Write down exactly what each phone sees afterwards. That is the problem a DBMS solves.

Deliverable

The five-app worksheet, plus a short paragraph comparing a database and a DBMS in your own words.

02 Build your database lab

VIDEO 2 · 60 MIN

OBJECTIVE Get a working PostgreSQL server, two ways to talk to it, the shopfloor dataset, and a reset you trust.

STEPS

1. Install Docker Desktop (or Docker Engine on Linux) and confirm docker run hello-world works.
2. Start PostgreSQL 16 in a container with a named volume, so your data survives restarts.
3. Connect with psql from the command line, then connect again from DBeaver or pgAdmin.
4. Create the shopfloor database and load shopfloor.sql from the course files.
5. Run your first query and compare the row count with the dataset table at the front of this guide.
6. Write reset.sh (Appendix A) and run it once.
7. Create labnotes.md and record your connection details, versions, and anything that went wrong.

```
docker run --name dblab -e POSTGRES_PASSWORD=lab \
  -p 5432:5432 -v dblab-data:/var/lib/postgresql/data \
  -d postgres:16

psql -h localhost -U postgres -c "CREATE DATABASE shopfloor;"
psql -h localhost -U postgres -d shopfloor -f shopfloor.sql
psql -h localhost -U postgres -d shopfloor \
  -c "SELECT count(*) FROM orders;"
```

Install, load, and run a first query. The password is for the lab only.

CHECKPOINT

SELECT count(*) FROM orders; returns 60000 in both psql and your graphical client.

BREAK IT

Run DROP TABLE orders CASCADE; and confirm queries now fail. Then run reset.sh and check that the count is back to 60000.

Deliverable reset.sh, plus the first labnotes.md entry with connection string, PostgreSQL version, and client versions.

03 First queries with SELECT

VIDEO 3 · 60 MIN

OBJECTIVE Answer specific questions by choosing columns, filtering, sorting, limiting, and removing duplicates.

STEPS

1. List the name and city of every customer. Then list only customers in one country.
2. Find active products in categories 3 or 7 that cost more than 50. Write the filter with AND, OR, and parentheses.
3. Sort that result by price, highest first, then show only the top 10.
4. List the distinct countries customers live in. Then list distinct (country, city) pairs, and explain why the count goes up.
5. Before running each query, write down the row count you expect. Explain every mismatch in your notebook.

```
SELECT name, unit_price
FROM products
WHERE discontinued = false
  AND (category_id = 3 OR category_id = 7)
  AND unit_price > 50
ORDER BY unit_price DESC
LIMIT 10;
```

Filtering, sorting, and limiting in one query.

CHECKPOINT

Every query has a predicted row count written next to it, and every mismatch has an explanation.

BREAK IT

Remove the parentheses around the OR condition and run the query again. Count the extra rows, then explain which rows got in and why AND binds tighter than OR.

Deliverable Eight queries, each with the question it answers, the predicted count, and the actual count.

04 Data types, expressions, and NULL

VIDEO 4 · 60 MIN

OBJECTIVE Compute new values from columns, and handle text, numbers, dates, and nulls without getting silently wrong answers.

STEPS

1. Run \d orders and \d order_items, and record the type of every column.
2. Compute a line total for each order item. Notice that some totals come back NULL.
3. Find out why, fix it with COALESCE, and check the grand total again.
4. Compare WHERE name = 'widget' with WHERE name ILIKE 'widget', and record the difference.
5. Sum 0.1 ten times as float and as numeric, and compare the two results.

6. Count the orders placed on 2026-03-01. Run SET TIME ZONE 'UTC' and then 'America/Los_Angeles', and compare the counts.
7. List unshipped orders with shipped_at IS NULL. Then explain why shipped_at = NULL returns no rows.

```
SELECT order_id,
       quantity * unit_price * (1 - discount) AS line_total
FROM order_items
LIMIT 20;

-- Bug hunt: why are some line totals NULL?
SELECT count(*) FROM order_items WHERE discount IS NULL;

-- Fix
SELECT sum(quantity * unit_price * (1 - coalesce(discount, 0)))
FROM order_items;
```

A null-propagation bug and its fix.

CHECKPOINT

The grand total has no nulls in it, and you can say in one sentence why a single NULL in an expression makes the whole result NULL.

BREAK IT

Write a revenue query that leaves out the COALESCE and check how far it differs from the correct total. That difference is real money a report would have lost without any error.

Deliverable A notebook table with one bug each for NULL, dates and time zones, and numeric precision: symptom, cause, and fix.

---

05 Think in sets

VIDEO 5 · 45 MIN

OBJECTIVE Turn a procedural solution into a declarative, set-based query, and trace how the query is logically evaluated.

STEPS

1. Write pseudocode for "find customers who have never ordered" as a loop over customers.
2. Rewrite it as one query. Try two versions, one with LEFT JOIN … IS NULL and one with NOT EXISTS.
3. Confirm both versions return the same count, then check the count a third way: total customers minus customers who have ordered.
4. Number each clause of the annotated query below by its logical evaluation order.
5. Try to use the alias n in the WHERE clause. Read the error and explain it using the evaluation order.

```
SELECT c.country, count(*) AS n      -- 5
FROM customers c                      -- 1
WHERE c.created_at >= '2025-01-01'    -- 2
GROUP BY c.country                    -- 3
HAVING count(*) > 50                  -- 4
ORDER BY n DESC                       -- 6
LIMIT 5;                              -- 7
```

Logical evaluation order. It is not the order you write the clauses in.

CHECKPOINT

Three independent methods agree on the number of customers who have never ordered.

BREAK IT

Trust a query without checking it: use an INNER JOIN to "find" customers with no orders and see it return zero. A query that silently returns a wrong answer is worse than one that errors.

Deliverable

The pseudocode, the two set-based queries, and an evaluation-order diagram.

PART II

Querying in depth

Joins, aggregation, subqueries, window functions, and changing data safely.

06 Join tables together

VIDEO 6 · 75 MIN

OBJECTIVE Answer questions that no single table can, using the right join type, and recognize when a join duplicates or loses rows.

STEPS

1. Join orders to customers to list each order with the customer's name.
2. Use a LEFT JOIN from customers to orders to keep customers with no orders, then compare the row count with the INNER JOIN.
3. Join order_items, products, and categories to show revenue lines with category names.
4. Self-join employees to list each person with their manager.
5. Leave out the join condition between orders and customers and work out the row count before running it with LIMIT.
6. Join orders to both order_items and payments in one query. Sum revenue and watch it inflate. Fix it by aggregating each table before joining.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON m.employee_id = e.manager_id
ORDER BY manager NULLS FIRST;
```

A self-join. The LEFT JOIN keeps the employee who has no manager.

CHECKPOINT

Revenue from your multi-table query matches the sum of order_items on its own, to the cent.

BREAK IT

Change a LEFT JOIN to an INNER JOIN in a "customers and their order counts" report. Find the customers who disappeared.

Deliverable Four join queries, plus a write-up of the payments fan-out bug: symptom, cause, and fix.

07 Aggregate and group

VIDEO 7 · 60 MIN

OBJECTIVE Produce summary reports that filter at the right stage and handle nulls correctly.

STEPS

1. Compute count, sum, avg, min, and max of order value across all orders.
2. Report revenue by category, then by category and month.
3. Exclude cancelled orders in WHERE, and keep only groups over 10,000 in HAVING. Explain why each filter belongs where it is.
4. Compare count(*) with count(shipped_at) and explain the difference.
5. Compare avg(discount) with avg(coalesce(discount, 0)) and decide which one the business actually means.
6. Optionally, add GROUP BY ROLLUP (category, month) for subtotals.

```
SELECT cat.name AS category,
       date_trunc('month', o.ordered_at) AS month,
       sum(oi.quantity * oi.unit_price) AS revenue
FROM order_items oi
JOIN orders o       ON o.order_id = oi.order_id
JOIN products p     ON p.product_id = oi.product_id
JOIN categories cat ON cat.category_id = p.category_id
WHERE o.status <> 'cancelled'
GROUP BY cat.name, month
HAVING sum(oi.quantity * oi.unit_price) > 10000
ORDER BY month, revenue DESC;
```

Filter rows before grouping (WHERE) and filter groups after (HAVING).

CHECKPOINT

Your category totals add up to the overall non-cancelled revenue.

BREAK IT

Put the revenue threshold in WHERE instead of HAVING and read the error. Then report "average discount" without handling NULL and calculate how far off it is.

Deliverable A monthly category revenue report, plus notes on each null behavior you observed.

08 Subqueries and nested logic

VIDEO 8 · 60 MIN

OBJECTIVE Solve problems that need the answer to one question inside another, and weigh subqueries against joins.

STEPS

1. List products priced above the average price, using a scalar subquery.
2. List customers who bought from category 4, using EXISTS.
3. For each customer, show their latest order date with a correlated subquery.
4. Rewrite the latest-order query as a join with GROUP BY, and confirm the results match.
5. Run EXPLAIN ANALYZE on both versions and record the timings.
6. Write a scalar subquery that returns more than one row, and read the error.

```
SELECT c.customer_id, c.name
FROM customers c
WHERE EXISTS (
  SELECT 1
  FROM orders o
  JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.customer_id = c.customer_id
    AND oi.product_id IN (SELECT product_id
                          FROM products
                          WHERE category_id = 4)
);
```

An existence check with a nested list subquery.

CHECKPOINT

The subquery version and the join version return identical rows. Check with EXCEPT in both directions.

BREAK IT

Find customers who were never referred with NOT IN (SELECT referred_by FROM customers). It returns nothing, because referred_by contains NULLs. Fix it with NOT EXISTS.

Deliverable Paired subquery and join solutions with timings, and a note on which one reads better.

09 Window functions and CTEs

VIDEO 9 · 75 MIN

OBJECTIVE Rank within categories, compute running and moving totals, compare rows with their neighbors, and walk a hierarchy.

STEPS

1. Rank products by revenue within each category using ROW_NUMBER, RANK, and DENSE_RANK side by side.
2. Keep only the top three per category, using a CTE.
3. Compute daily revenue with a running total and a 7-day moving average.
4. Use LAG to show month-over-month change in revenue.
5. Write a recursive CTE that lists every employee under one manager, with their depth in the hierarchy.
6. Write a second recursive CTE that prints the full category path, for example Garden > Tools > Pruners.

```
WITH daily AS (
  SELECT o.ordered_at::date AS day,
         sum(oi.quantity * oi.unit_price) AS revenue
  FROM orders o
  JOIN order_items oi ON oi.order_id = o.order_id
  GROUP BY day
)
SELECT day, revenue,
       sum(revenue) OVER (ORDER BY day)          AS running_total,
       avg(revenue) OVER (ORDER BY day
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS avg_7d
FROM daily
ORDER BY day;
```

Running and sliding windows over a named CTE step.

CHECKPOINT

The last running_total equals total revenue, and your hierarchy query returns every employee exactly once.

BREAK IT

In a copy of employees, set one person to be their own manager's manager, which creates a cycle. Run the recursive query and watch it fail to finish. Fix it with a depth limit or PostgreSQL's CYCLE clause.

Deliverable Four analytical queries, plus a sketch explaining the default window frame.

10 Modify data safely

VIDEO 10 · 60 MIN

OBJECTIVE Insert, update, and delete with the test-first habit, and see firsthand what an unfiltered update does.

STEPS

1. Create a working copy: CREATE TABLE products_copy AS SELECT * FROM products;
2. Insert one row, then insert several rows with a single multi-row INSERT.

3. Before any UPDATE or DELETE, run the same WHERE as a SELECT count(*), and write the number down.
4. Inside BEGIN, apply a 5% price increase to category 5. Check that the reported row count matches your number, then COMMIT.
5. Delete discontinued products from the copy. Compare DELETE with TRUNCATE.
6. Use INSERT … SELECT to archive old orders into an orders_archive table, and UPDATE … FROM to reprice items using a lookup table.

```
BEGIN;
SELECT count(*) FROM products_copy
WHERE category_id = 5;                -- expect: ____

UPDATE products_copy
SET unit_price = unit_price * 1.05
WHERE category_id = 5;                -- UPDATE n must match

SELECT * FROM products_copy WHERE category_id = 5 LIMIT 5;
COMMIT;                               -- or ROLLBACK;
```

Test first, change inside a transaction, check, then commit.

CHECKPOINT

Every modification in your notebook has a predicted count that matches the count the database reported.

BREAK IT

On the copy only, run UPDATE products_copy SET unit_price = 0; with no WHERE. Read "UPDATE 400", then rebuild the copy from products.

Deliverable A personal checklist to run before any modification, and an archive script.

---

11 Views, patterns, and query craft

VIDEO 11 · 60 MIN

OBJECTIVE Save reusable queries as views, apply standard patterns, and refactor a tangled query into one people can maintain.

STEPS

1. Create an order_totals view, then a customer_value view built on top of it.
2. Run EXPLAIN on a query against customer_value and look at how much work the stacked views hide.
3. Find duplicate customer emails, ignoring case, with the GROUP BY … HAVING pattern.
4. Get the top product per category, once with DISTINCT ON and once with a window function.
5. Open tangled.sql from the course files and refactor it into named CTEs with consistent formatting and clear aliases.
6. Write a check query that proves the refactored version returns the same result as the original.

```
CREATE VIEW order_totals AS
SELECT o.order_id, o.customer_id, o.ordered_at,
       sum(oi.quantity * oi.unit_price
           * (1 - coalesce(oi.discount, 0))) AS total
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_id;

-- Duplicate pattern
SELECT lower(email) AS email, count(*)
FROM customers
GROUP BY 1
HAVING count(*) > 1;
```

A reusable view and the find-duplicates pattern.

CHECKPOINT

(original EXCEPT refactored) and (refactored EXCEPT original) both return zero rows.

BREAK IT

Filter customer_value to a single customer and read the plan. Note how much of the database it still touches, because a view runs its full query every time.

Deliverable

Two views, the refactored query, and your own SQL style rules (five lines at most).

PART III

Database design

Designing schemas that keep data correct before the first query is written.

12 Design before you build

VIDEO 12 · 60 MIN

OBJECTIVE Turn a real scenario into entities, attributes, and relationships, without creating a single table.

STEPS

1. Scenario: a community tool library. Members borrow tools, reserve them ahead of time, and report damage. There are several branches.
2. Write the ten questions the application must be able to answer, for example "Which tools are overdue at branch X?"
3. Underline the nouns in your questions to get candidate entities. Merge synonyms and drop anything that is really an attribute.
4. List the attributes for each entity, and describe each relationship with a verb, such as "member borrows tool".
5. Open tool_library.csv from the course files, the spreadsheet this system replaces, and list every fact it stores more than once.
6. Check each of your ten questions against the design and note which entities would answer it.

```
Entity:       _____________
Represents:   (the real-world thing)
Attributes:   _____________
Identified by: _____________
Relates to:   _____________  via verb: ________
Answers Qs:   #__, #__
```

Entity worksheet. Use one per entity.

CHECKPOINT

All ten questions can be answered from your entities, and no fact appears in more than one entity.

BREAK IT

Sketch the "design" you would get by copying the spreadsheet columns into one table. Write down one way that table could end up saying two contradictory things.

Deliverable The ten questions, the entity worksheets, and a redundancy list for the spreadsheet.

13 Model relationships

VIDEO 13 · 75 MIN

OBJECTIVE Draw an entity-relationship diagram and translate it into related tables.

STEPS

1. Draw the tool library ERD with crow's-foot notation in draw.io, dbdiagram, or on paper.
2. Label each relationship 1:1, 1:N, or M:N, and say whether it is optional or mandatory on each side.
3. Resolve each M:N relationship, such as tools and tags, with a junction table.
4. Decide whether any 1:1 relationship deserves its own table, for example a member's private contact details.
5. Create a toollib schema and write CREATE TABLE statements that match the diagram.

```
CREATE SCHEMA toollib;
SET search_path = toollib;

CREATE TABLE members (member_id int PRIMARY KEY, name text, email text);
CREATE TABLE tools   (tool_id   int PRIMARY KEY, name text, branch_id int);
CREATE TABLE tags    (tag_id    int PRIMARY KEY, label text);
CREATE TABLE tool_tags (                        -- M:N junction
  tool_id int, tag_id int,
  PRIMARY KEY (tool_id, tag_id)
);
CREATE TABLE loans (
  loan_id int PRIMARY KEY, tool_id int, member_id int,
  borrowed_at timestamptz, due_at timestamptz, returned_at timestamptz
);
```

First pass at the tables. Constraints come in Lab 14.

CHECKPOINT

Every line on the diagram corresponds to a column or a junction table, and every table appears on the diagram.

BREAK IT

Store tags as a comma-separated text column on tools. Try to write "all tools tagged garden" and "rename the garden tag", and record both problems.

Deliverable — The ERD image and toollib_v1.sql.

14 Keys and constraints

VIDEO 14 · 60 MIN

OBJECTIVE Make the database enforce correctness on its own, and watch it refuse invalid data.

STEPS

1. Confirm each table has a primary key. Decide surrogate or natural key for tools, and write down why.
2. Add foreign keys from loans to members and tools, and from tool_tags to both sides. Choose an ON DELETE behavior for each.
3. Add UNIQUE on members.email, NOT NULL wherever a value is required, and CHECK (due_at > borrowed_at).
4. Try six invalid inserts: a duplicate email, a loan for a tool that doesn't exist, a due date before the borrow date, a missing name, a duplicate tag, and a duplicate primary key.
5. Try to delete a member who has loans and record what happens.

```
ALTER TABLE loans
  ADD CONSTRAINT loans_tool_fk   FOREIGN KEY (tool_id)   REFERENCES tools,
  ADD CONSTRAINT loans_member_fk FOREIGN KEY (member_id) REFERENCES members
      ON DELETE RESTRICT,
  ADD CONSTRAINT loans_dates_ck  CHECK (due_at > borrowed_at),
  ALTER COLUMN borrowed_at SET NOT NULL;

INSERT INTO loans VALUES (1, 9999, 1, now(), now() + interval '7 days', NULL);
-- ERROR: violates foreign key constraint "loans_tool_fk"
```

Named constraints make the error messages readable.

CHECKPOINT

All six invalid inserts are rejected, and each error message names the constraint that caught it.

BREAK IT

Drop loans_tool_fk, insert three loans for tool 9999, and put the constraint back. Read the error, then write a query that finds the orphaned rows.

Deliverable toollib_v2.sql with all constraints, plus a table of each rejected insert and its error.

---

15 Normalize a redundant table

VIDEO 15 · 75 MIN

OBJECTIVE Find the anomalies in a flat table and normalize it into related tables by reasoning about where each fact belongs.

STEPS

1. Load loans_flat from the course files. Each row repeats the member, tool, and branch details.
2. Find real examples of the anomalies: two phone numbers for one member (update), no way to add a tool nobody has borrowed (insert), and a member who vanishes when their last loan is deleted (delete).
3. Write the functional dependencies, for example member_email → member_name, member_phone.
4. Get to 1NF, then 2NF, then 3NF, writing one sentence on what each step fixes.

5. Migrate the data into the new tables with INSERT ... SELECT DISTINCT.
6. Rebuild the flat view with joins and compare it with the original.

```
-- Evidence of drift: one member, several phone numbers
SELECT member_email, count(DISTINCT member_phone) AS phones
FROM loans_flat
GROUP BY member_email
HAVING count(DISTINCT member_phone) > 1;
```

Redundancy shows up as facts that disagree.

CHECKPOINT

The joined, normalized tables reproduce loans_flat exactly. EXCEPT in both directions returns zero rows, apart from the drifted values you resolved.

BREAK IT

Normalize branch addresses down to separate street, city, and postcode tables. Write a simple "loans by city" query and count the joins it now needs. Decide where to stop.

Deliverable An anomaly report, the dependency list, and the migration script.

---

16 Denormalize on purpose

VIDEO 16 · 75 MIN

OBJECTIVE Add controlled redundancy for a slow query and build the mechanism that keeps it consistent.

STEPS

1. Back in shopfloor, time a "customer lifetime value and order count" query with EXPLAIN ANALYZE.
2. Add lifetime_value and order_count columns to customers and backfill them.
3. Write a trigger on order_items that keeps lifetime_value correct on INSERT, UPDATE, and DELETE.
4. Test it: insert, change, and delete items, and check the stored value after each step.
5. Write a drift-detection query that compares the stored value with a fresh calculation.
6. Decide whether the speedup justifies the risk, and record your reasoning.

```
CREATE FUNCTION bump_ltv() RETURNS trigger AS $$
BEGIN
  UPDATE customers c
  SET lifetime_value = lifetime_value
      + coalesce(NEW.quantity * NEW.unit_price, 0)
      - coalesce(OLD.quantity * OLD.unit_price, 0)
  FROM orders o
  WHERE o.order_id = coalesce(NEW.order_id, OLD.order_id)
    AND c.customer_id = o.customer_id;
  RETURN NULL;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER order_items_ltv
AFTER INSERT OR UPDATE OR DELETE ON order_items
FOR EACH ROW EXECUTE FUNCTION bump_ltv();
```

Redundancy with a plan to keep it consistent.

CHECKPOINT

After twenty random changes, the drift-detection query returns zero rows.

BREAK IT

Disable the trigger, make five changes, and enable it again. Run the drift query to find the damage, then fix it with a backfill.

Deliverable Before and after timings, the trigger, the drift query, and a one-paragraph decision record.

---

17 Design for a messy scenario

VIDEO 17 · 120 MIN

OBJECTIVE Design, name, and document a complete database for realistic requirements full of exceptions.

STEPS

1. Scenario: a network of veterinary clinics. A pet can have several owners, staff work at several clinics, insurance is optional, some attributes only apply to some species, prices change over time, and the system must keep the price that was actually charged.
2. List every awkward case before you design anything.
3. Write naming conventions (singular or plural, how keys are named, how timestamps are named) and follow them.
4. Build the vetclinic schema with keys and constraints.
5. Document every table and column with COMMENT ON, then generate a data dictionary from the catalog.
6. Hold a design review against twelve questions the clinic needs answered. Then write the v2 migration for one new requirement: telemedicine appointments.

```
COMMENT ON TABLE  vetclinic.appointment IS
  'A booked visit. One pet, one clinic, one or more staff.';
COMMENT ON COLUMN vetclinic.appointment.charged_price IS
  'Price at time of visit, copied from price_list; never recalculated.';

SELECT c.table_name, c.column_name, c.data_type,
       col_description((c.table_schema||'.'||c.table_name)::regclass,
                       c.ordinal_position) AS meaning
FROM information_schema.columns c
WHERE c.table_schema = 'vetclinic'
ORDER BY c.table_name, c.ordinal_position;
```

The documentation lives in the database, and you generate the dictionary from it.

CHECKPOINT

Every column has a comment, all twelve review questions have a working query, and the v2 migration runs without rebuilding any table.

BREAK IT

Give your schema to a classmate (or yourself a week later) without any explanation. Every name they have to ask about is a naming failure. Rename those.

Deliverable

vetclinic.sql, the naming conventions, the generated data dictionary, the design review, and the v2 migration.

PART IV

The engine and performance

What the engine does, how to make it fast, and how it stays correct under load.

18 Look under the hood

VIDEO 18 · 60 MIN

OBJECTIVE See how PostgreSQL stores, caches, and logs data when you query and modify it.

STEPS

1. Find the file behind the orders table with pg_relation_filepath, and check its size and page count.
2. Look at ctid for a few rows. Update one row, then look at its ctid again, and explain why it moved.
3. Run EXPLAIN (ANALYZE, BUFFERS) on a full scan twice. Compare shared read (from disk) with shared hit (from cache).
4. Check shared_buffers, and estimate how much of shopfloor fits in memory.
5. Record pg_current_wal_lsn(), run a bulk update inside a transaction, commit, and measure how much WAL it wrote.

```
SELECT pg_relation_filepath('orders'),
       pg_size_pretty(pg_relation_size('orders')) AS size,
       pg_relation_size('orders') / 8192        AS pages;

SELECT ctid, order_id, status FROM orders WHERE order_id = 42;
UPDATE orders SET status = status WHERE order_id = 42;
SELECT ctid, order_id, status FROM orders WHERE order_id = 42;

EXPLAIN (ANALYZE, BUFFERS) SELECT count(*) FROM order_items;
```

Physical storage, row versions, and the buffer cache.

CHECKPOINT

You can explain, with numbers from your own run, why the second scan was faster and why the updated row moved.

BREAK IT

Assume an UPDATE changes a row in place. Update every row of a copy of orders, check the table size before and after, then run VACUUM and check again.

Deliverable An annotated diagram of a query and a write moving through cache, data files, and WAL.

19 Make a slow query fast with an index

VIDEO 19 · 75 MIN

OBJECTIVE Add the right index, measure the improvement, and understand when an index is not used.

STEPS

1. Turn on \timing. Run EXPLAIN ANALYZE on "orders for customer 1234 in the last year" and record the time and the scan type.
2. Create an index on customer_id and measure again.
3. Replace it with a composite index on (customer_id, ordered_at). Test filtering on each column alone and explain why column order matters.
4. Find three queries that ignore an index: lower(email) = ..., name LIKE '%drill', and status = 'shipped' when most rows are shipped.
5. Fix the first with an expression index. Explain why the other two are reasonable to leave alone.
6. Time a 50,000-row insert into a copy of orders with zero indexes and with five, and record the write cost.

```
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE customer_id = 1234
  AND ordered_at >= now() - interval '1 year';

CREATE INDEX orders_cust_date_ix ON orders (customer_id, ordered_at);

CREATE INDEX customers_email_lower_ix ON customers (lower(email));
```

A composite index and an expression index.

CHECKPOINT

The customer query switches from Seq Scan to Index Scan and runs at least ten times faster.

BREAK IT

Index every column of a copy of order_items and rerun the bulk insert. Then record the total index size with pg_indexes_size.

Deliverable A before-and-after table with time, scan type, and rows examined, plus the write-cost measurements.

20 Read execution plans

VIDEO 20 · 75 MIN

OBJECTIVE Read a plan, find the slow operation, and confirm in the plan that your fix worked.

STEPS

1. Run EXPLAIN and EXPLAIN (ANALYZE, BUFFERS) on the five queries in plans.sql from the course files.
2. For each one, circle the most expensive node and name the operation: Seq Scan, Index Scan, Hash Join, Nested Loop, or Sort.
3. Compare estimated rows with actual rows. A gap of ten times or more is a lead worth following.
4. Create stale statistics: copy order_items, turn autovacuum off for the copy, insert a million rows, and watch the estimates go wrong. Run ANALYZE and compare.
5. Improve each query by adding an index or rewriting it, and confirm the change in the new plan.

```
ALTER TABLE items_copy SET (autovacuum_enabled = false);
INSERT INTO items_copy SELECT * FROM order_items, generate_series(1, 5);

EXPLAIN ANALYZE SELECT * FROM items_copy WHERE product_id = 17;
-- rows=... (estimate) vs actual rows=...

ANALYZE items_copy;
EXPLAIN ANALYZE SELECT * FROM items_copy WHERE product_id = 17;
```

Stale statistics make for bad estimates.

CHECKPOINT

All five queries are faster, and each improvement shows up as a changed node in the plan, not just a lower time.

BREAK IT

Rewrite a query because you think it will be faster, without looking at the plan. Then check the plan. Record whether you were right.

Deliverable Five before-and-after plan excerpts with one-line diagnoses.

---

21 Transactions and integrity

VIDEO 21 · 60 MIN

OBJECTIVE Make a multi-step operation all-or-nothing, and show the damage when it isn't.

STEPS

1. Create an accounts table with a CHECK (balance >= 0) constraint and two accounts.
2. Without a transaction, transfer 100 with two separate UPDATEs, and make the second one fail. Check the balances.
3. Repeat inside BEGIN ... COMMIT and confirm the failure rolls back both changes.
4. Use a SAVEPOINT to undo part of a transaction.
5. Do the same transfer from Python with psycopg, using a transaction block.
6. Leave a session open after BEGIN and find it in pg_stat_activity as "idle in transaction".

```
import psycopg

with psycopg.connect("dbname=shopfloor user=postgres") as conn:
    with conn.transaction():
        conn.execute("UPDATE accounts SET balance = balance - %s WHERE id = %s", (100, 1))
        conn.execute("UPDATE accounts SET balance = balance + %s WHERE id = %s", (100, 2))
    # leaving the block commits; an exception rolls back
```

Application code that gets atomicity right.

CHECKPOINT

Total money across all accounts is identical after every test, whether it succeeded or failed.

BREAK IT

Run the two-UPDATE version with a deliberate failure and calculate how much money disappeared or was created.

Deliverable A short write-up of atomicity, consistency, isolation, and durability, each illustrated by something you saw in this lab.

---

22 Concurrency, locking, and isolation

VIDEO 22 · 90 MIN

OBJECTIVE Reproduce a lost update and a deadlock, then fix each one.

STEPS

1. Open two psql sessions, A and B, side by side.
2. Lost update: both sessions read the balance, both calculate a new value in the application, and both write it. One change is lost.
3. Fix it three ways: a single atomic UPDATE, SELECT ... FOR UPDATE, and REPEATABLE READ with a retry when you get error 40001.
4. Deadlock: A locks row 1 and then row 2, while B locks row 2 and then row 1. Read the deadlock error.
5. Fix it by always locking rows in the same order.
6. Add a version column and do an optimistic update that fails when the version has changed.

```
-- Session A                        -- Session B
BEGIN;                              BEGIN;
UPDATE accounts SET balance=balance-1
 WHERE id = 1;
                                    UPDATE accounts SET balance=balance-1
                                     WHERE id = 2;
UPDATE accounts SET balance=balance+1
 WHERE id = 2;     -- waits
                                    UPDATE accounts SET balance=balance+1
                                     WHERE id = 1;
                                    -- ERROR: deadlock detected
```

Run the two columns step by step, top to bottom.

CHECKPOINT

Under each fix, 100 concurrent increments (from a script or pgbench) produce exactly +100.

BREAK IT

Run the lost-update test at READ COMMITTED and SERIALIZABLE, and record how many updates were lost and how many retries each needed.

Deliverable A timeline diagram for each problem, plus your rule for choosing an isolation level.

---

23 Replication and read scaling

VIDEO 23 · 90 MIN

OBJECTIVE Set up a read replica, send reads to it, and measure the replication lag that comes with it.

STEPS

1. Start a primary and a streaming replica with Docker Compose (compose file in the course files).
2. On the primary, confirm the replica is connected with pg_stat_replication.
3. Write on the primary and read on the replica, and record how long it takes for the write to appear.
4. Try to write on the replica and read the error.
5. Pause replay on the replica, keep writing on the primary, and see stale reads directly.
6. Shard-key thought exercise: compare how evenly orders spread across four shards keyed by country and keyed by hash(customer_id).

```
-- On the replica
SELECT now() - pg_last_xact_replay_timestamp() AS lag;
SELECT pg_wal_replay_pause();     -- simulate a lagging replica
SELECT pg_wal_replay_resume();

-- Shard-skew check (on the primary)
SELECT abs(hashtext(customer_id::text)) % 4 AS shard, count(*)
FROM orders GROUP BY shard ORDER BY shard;
```

Measuring lag and checking how evenly a shard key spreads data.

CHECKPOINT

You have a measured lag figure under normal load and under paused replay, plus a shard distribution for each candidate key.

BREAK IT

Write a "place order, then show order history" flow that reads from the replica, and catch the moment the new order is missing.

Deliverable

The compose file, the lag measurements, and a short recommendation for which reads can safely go to replicas.

PART V

Beyond the single relational database

Other data models, analytics, cloud deployment, and modern application architecture.

24 Relational versus document

VIDEO 24 · 75 MIN

OBJECTIVE Model the same orders in a relational and a document database, and match each model to the access patterns it suits.

STEPS

1. Start MongoDB in a container. If you would rather not, use a PostgreSQL jsonb column instead.
2. Export 1,000 orders as documents with the customer and items embedded.
3. Run the same four access patterns on both: fetch one full order, revenue by category, rename a product everywhere, and all orders for one customer.
4. Count the queries or statements each one takes and time them.
5. Note which integrity rules the document model no longer enforces for you.

```
db.orders.insertOne({
  _id: 42,
  customer: { id: 1234, name: "R. Okafor", city: "Leeds" },
  ordered_at: ISODate("2026-03-01T10:15:00Z"),
  items: [
    { sku: "PR-118", name: "Bypass pruner", qty: 2, price: 24.50 },
    { sku: "GL-004", name: "Work gloves",   qty: 1, price: 9.00 }
  ]
});

db.orders.aggregate([
  { $unwind: "$items" },
  { $group: { _id: "$items.sku",
      revenue: { $sum: { $multiply: ["$items.qty", "$items.price"] } } } }
]);
```

An embedded order document and an aggregation across documents.

CHECKPOINT

A four-row comparison table showing which model fits each access pattern better, with your measurements.

BREAK IT

Rename a product in one document but not in the others. Then try to find out which name is correct.

Deliverable The comparison table, plus one paragraph on when you would choose each model.

---

25 Build a database selection framework

VIDEO 25 · 45 MIN

OBJECTIVE Turn the course so far into a reusable framework for matching problems to database models.

STEPS

1. Define the criteria: data shape, relationships, consistency needs, access patterns, realistic scale (actual numbers), and the team's capacity to run it.
2. For each criterion, write the question to ask and what the answer points to.
3. Score five scenarios: payroll, a web session store, a product catalog with very different attributes, social recommendations, and IoT sensor telemetry.
4. For each scenario, name the primary database and any secondary stores, and explain why.
5. Write one decision record in the format below.

```
DECISION: _____________________
Context:      what the data is and how it's used
Scale today:  __ rows, __ writes/s, __ reads/s
Scale in 2y:  __ (evidence: _________)
Consistency:  must always agree? Y / N — which facts?
Chosen:       _________   Rejected: _________ because _________
Ops cost:     who runs it, backup, upgrades, on-call
Revisit when: _________
```

Decision record template, the takeaway artifact for this video.

CHECKPOINT

At least one scenario lands on "relational is enough", and every non-relational choice cites a measured or estimated scale number.

BREAK IT

Argue for the most fashionable option on the payroll scenario. Then list the integrity rules you would have to rebuild in application code.

Deliverable The selection framework (one page), five scored scenarios, and one decision record.

26 Build a small data warehouse

VIDEO 26 · 90 MIN

OBJECTIVE Load data into a dimensional model and see why it outperforms the transactional design on analytical queries.

STEPS

1. Create a dw schema with fact_sales, dim_date, dim_product, and dim_customer.
2. Write the load step as INSERT ... SELECT from shopfloor, and make it safe to run twice.
3. Reconcile the load: fact revenue must equal the source revenue.
4. Run "revenue by quarter, category, and country" against shopfloor and against dw, and compare the plans and timings.
5. Export fact_sales to Parquet and run the same query in DuckDB to see columnar storage at work.

```
CREATE TABLE dw.fact_sales (
  date_key     int REFERENCES dw.dim_date,
  product_key  int REFERENCES dw.dim_product,
  customer_key int REFERENCES dw.dim_customer,
  quantity     int,
  revenue      numeric(12,2)
);

-- DuckDB
SELECT d.quarter, p.category, sum(f.revenue)
FROM 'fact_sales.parquet' f
JOIN 'dim_date.parquet' d USING (date_key)
JOIN 'dim_product.parquet' p USING (product_key)
GROUP BY ALL ORDER BY ALL;
```

A star-schema fact table and a columnar query.

CHECKPOINT

Source and warehouse revenue match exactly, and running the load twice doesn't create duplicates.

BREAK IT

Run the analytical query against shopfloor while a pgbench workload is writing to it, and watch transaction latency rise.

Deliverable A star-schema diagram, the load script, the reconciliation query, and a three-way timing comparison.

27 Run a managed database and ship a schema change

VIDEO 27 · 120 MIN

OBJECTIVE Deploy a managed cloud database and apply a schema change through a pipeline without breaking the running application.

STEPS

1. Create the smallest managed PostgreSQL instance your provider offers. Set a budget alert first.
2. Record what the provider handles for you (patching, backups, point-in-time recovery, failover) and what it doesn't (logical errors, retention beyond its limit, testing restores).
3. Restrict network access to your own IP and require TLS.
4. Load a subset of shopfloor, and run a small script that queries it every second to stand in for a live app.
5. Rename customers.city to locality using expand-and-contract: add the new column, backfill it, switch the app, then drop the old column. Ship each step as a separate migration through a CI pipeline.
6. Tear down the instance when you are done.

```
# .github/workflows/migrate.yml (excerpt)
- name: Apply migrations
  run: flyway -url=$DB_URL -user=$DB_USER -password=$DB_PASS migrate

-- V7__add_locality.sql      (expand)
ALTER TABLE customers ADD COLUMN locality text;
-- V8__backfill_locality.sql
UPDATE customers SET locality = city WHERE locality IS NULL;
-- V9__drop_city.sql         (contract, after app deploy)
ALTER TABLE customers DROP COLUMN city;
```

Expand and contract, one migration per pipeline run.

CHECKPOINT

The polling script logs no errors at any point during the three migrations.

BREAK IT

Apply a plain RENAME COLUMN while the polling script is running and record how long the "app" is broken.

Deliverable A provider responsibility list, the migration files, the pipeline config, and the polling logs.

28 Look behind the ORM and split data across services

VIDEO 28 · 75 MIN

OBJECTIVE See the SQL an abstraction layer actually generates, and model data owned by separate services.

STEPS

1. Map customers and orders in SQLAlchemy with echo=True.
2. Loop over 50 customers and print how many orders each has. Count the SQL statements that get logged.
3. Fix the N+1 pattern with selectinload and count the statements again.
4. Design a split into two services, orders and inventory, each with its own database. Write down which service owns which tables.
5. Add an outbox table to the orders service so "order placed" events are written in the same transaction as the order.

```
engine = create_engine("postgresql+psycopg://postgres:lab@localhost/shopfloor",
                       echo=True)

with Session(engine) as s:
    for c in s.scalars(select(Customer).limit(50)):
        print(c.name, len(c.orders))          # 1 + 50 queries

    stmt = select(Customer).options(selectinload(Customer.orders)).limit(50)
    for c in s.scalars(stmt):
        print(c.name, len(c.orders))          # 2 queries
```

The N+1 query pattern and its fix.

CHECKPOINT

The statement count drops from 51 to 2, and the outbox row and the order row commit or roll back together.

BREAK IT

In the two-service design, place an order that reserves stock in inventory, and crash between the two writes. Describe the inconsistent state you're left with and how the outbox prevents it.

Deliverable Logged SQL before and after, a service data-ownership map, and the outbox schema.

PART VI

Operations, security, and troubleshooting

Keeping databases recoverable, secure, and diagnosable.

29 Back up, restore, and monitor

VIDEO 29 · 90 MIN

OBJECTIVE Prove you can restore, put basic monitoring in place, and recover from a simulated failure.

STEPS

1. Take a backup with pg_dump -Fc.
2. Restore it into a new database called shopfloor_restore and compare row counts and revenue totals with the original.
3. Enable pg_stat_statements and write monitoring queries for active connections, cache hit ratio, the slowest statements, and the largest tables.
4. Run VACUUM (ANALYZE) and compare the statistics before and after.
5. Simulate a disaster by dropping shopfloor. Restore it from the backup and time the whole recovery, which is your recovery time objective (RTO).

```
pg_dump -h localhost -U postgres -Fc shopfloor > shopfloor.dump
createdb -h localhost -U postgres shopfloor_restore
pg_restore -h localhost -U postgres -d shopfloor_restore shopfloor.dump

-- Cache hit ratio
SELECT round(100.0 * sum(blks_hit) / nullif(sum(blks_hit + blks_read), 0), 2)
FROM pg_stat_database;
```

A backup you haven't restored is only a hope.

CHECKPOINT

The restored database matches the original on row counts and revenue, and your RTO is written down as a measured number.

BREAK IT

Take a backup that silently leaves something out, for example with —schema-only or by excluding one table. Restore it, and notice how long it takes to find what's missing without a verification step.

Deliverable A backup-and-restore runbook, the monitoring queries, and the RTO measurement.

30 Least privilege and SQL injection

VIDEO 30 · 90 MIN

OBJECTIVE Lock down access, exploit an injectable query in your own lab, and close the hole with parameterization.

STEPS

1. Create roles: app_read (SELECT only), app_write (only the tables the app writes), and a reporting login with no access to payments.
2. Revoke the default privileges from PUBLIC and confirm each role is refused outside its grants.
3. Check that password_encryption is scram-sha-256, and check whether your connection uses SSL.
4. Run vulnerable.py from the course files. Its search builds SQL by string formatting.
5. Inject ' OR '1'='1 to dump every customer, then use a UNION payload to read emails from another table.
6. Fix it with parameterized queries, rerun every payload, and confirm they now fail. Then turn on statement logging for changes.

```
# Vulnerable: input becomes SQL
cur.execute(f"SELECT name FROM customers WHERE city = '{city}'")

# Fixed: input stays data
cur.execute("SELECT name FROM customers WHERE city = %s", (city,))

-- Least privilege
CREATE ROLE app_read NOLOGIN;
GRANT SELECT ON customers, orders, products TO app_read;
CREATE ROLE reporting LOGIN PASSWORD '...' IN ROLE app_read;
```

Only attack systems you own. This lab runs entirely on your own machine.

CHECKPOINT

Every injection payload returns zero rows against the fixed code, and the reporting login gets "permission denied" on payments.

BREAK IT

Connect the app as postgres, a superuser, and run the injection again. Write down what an attacker could now do beyond reading data.

Deliverable A role and grant matrix, the before-and-after code, the payload log, and an audit-log sample.

31 Troubleshoot staged incidents

VIDEO 31 · 120 MIN

OBJECTIVE Diagnose five staged problems at the right layer and fix each root cause.

STEPS

1. Run the incident scripts from the course files one at a time. Reset between incidents.
2. Incident 1, wrong results: a revenue report is 18% too high.
3. Incident 2, slow query: an account page that used to load instantly now takes several seconds.
4. Incident 3, blocking: checkout hangs intermittently, then occasionally fails with a deadlock.
5. Incident 4, availability: the app reports "too many connections".
6. Incident 5, integrity: order counts per customer don't match between two reports.
7. For each one, record the symptom, your hypothesis, the evidence, the root cause, the fix, and how to prevent it.

```
-- Who is blocking whom
SELECT blocked.pid  AS blocked_pid,  blocked.query,
       blocking.pid AS blocking_pid, blocking.query AS blocking_query
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocking
  ON blocking.pid = ANY (pg_blocking_pids(blocked.pid));

-- Connections by state
SELECT state, count(*) FROM pg_stat_activity GROUP BY state;
```

Two diagnostic queries you will reach for again and again.

CHECKPOINT

All five root causes are identified with evidence (a plan, a lock graph, a count), not by guessing.

BREAK IT

For one incident, apply the first fix that comes to mind before gathering evidence. Note whether it fixed the root cause or only hid the symptom.

Deliverable

Five incident reports, plus your own troubleshooting checklist covering the query, the design, the engine, and the infrastructure.

CLOSING

Where to go next

Consolidate the course and commit to a direction.

32 Assess your skills and plan the next 90 days

VIDEO 32 · 60 MIN

OBJECTIVE Measure yourself against the whole course, choose a specialization, and commit to an ongoing practice project.

STEPS

1. Rate yourself from 1 to 4 on each part of the course, using your notebook entries as evidence.
2. Pick a specialization to explore: database administration, data engineering, database development, or performance and reliability engineering.
3. Choose one certification that fits that path and note what it shows employers.
4. Write a 90-day plan in three 30-day blocks, each with one measurable outcome.
5. Define an ongoing database project that you will design, query, tune, back up, and secure as it grows.

```
SPECIALIZATION: _____________________

Days 1-30   Consolidate   outcome: _____________________
Days 31-60  Deepen        outcome: _____________________
Days 61-90  Prove         outcome: _____________________

Ongoing project: _____________________
Weekly habit:    __ hours, every ________
Follow:          2 communities / 1 newsletter / 1 release log
```

90-day plan template.

CHECKPOINT

Every self-rating cites a lab, and every 30-day block ends in something someone else could check.

BREAK IT

Write the plan with no measurable outcomes, then try to tell on day 30 whether you're on track. Rewrite it.

Deliverable The self-assessment, the 90-day plan, and the project brief.

APPENDIX A

Reset and recovery

Use these whenever a lab leaves the environment in a state you don't trust. Resetting is part of the course, not a failure.

```
#!/usr/bin/env bash
set -euo pipefail
export PGHOST=localhost PGUSER=postgres PGPASSWORD=lab

dropdb --if-exists shopfloor
createdb shopfloor
psql -q -d shopfloor -f shopfloor.sql
psql -d shopfloor -c "SELECT count(*) AS orders FROM orders;"   # expect 60000
```

reset.sh, built in Lab 02. Keep it next to shopfloor.sql.

SYMPTOM	FIX
psql hangs, no prompt	An earlier transaction is still open. Run ROLLBACK; or check pg_stat_activity for sessions that are "idle in transaction".
"current transaction is aborted"	An error inside BEGIN puts the transaction in a failed state. Run ROLLBACK; and start again.
Container won't start: port 5432 in use	A local PostgreSQL is already running. Stop it, or map the container to -p 5433:5432.
Results differ from the guide	Your data has drifted from earlier labs. Run reset.sh and repeat the lab.
Lost the whole volume	docker volume rm dblab-data, then repeat the Lab 02 install. Your notebook and scripts live outside the container.

APPENDIX B

Lab notebook template

Copy this into labnotes.md for each lab. By Lab 32 the notebook doubles as your troubleshooting checklist and a portfolio.

```
## Lab __ — _____________________   date: ____

Question I was answering:

What I predicted:

What happened:

The failure I caused (Break it):
  symptom  →
  cause    →
  fix      →

One thing I'd check first next time:

Files: _____________________
```

One entry per lab. Keep it short and specific.